Rhizome Neural Operator

Technical Report

Haris Kapetanovic

Working draft — October 12, 2026

Abstract

[**TODO:** One paragraph: the problem (learning field-to-field maps for 21 cm reionization), what neural operators/FNOs give and miss, what the rhizome operator changes, and the headline result.]

Contents

1 Neural Operators and FNOs **3**
- 1.1 Operator learning: problem setup . . . 3
- 1.2 Neural operators as stacked kernel integrals . . . 3
- 1.3 The Fourier Neural Operator . . . 3
- 1.4 Discretisation invariance and its limits . . . 4
- 1.5 Limitations of a purely spectral kernel . . . 5

2 Spatial structure in neural operators and the U-Net **6**
- 2.1 Inductive biases: locality, scale and stationarity . . . 6
- 2.2 The U-Net . . . 7
- 2.3 Hybrid architectures: U-FNO and related . . . 9
- 2.4 Open problem: structure that adapts to the field . . . 9

3 Rhizome Neural Operator approach **10**
- 3.1 Design principles . . . 10
- 3.2 State-dependent integral interaction . . . 10
- 3.3 Gating: source and receiver factors . . . 10
- 3.4 Recurrent time-stepping . . . 11
- 3.5 Discretisation and spectral evaluation . . . 11
- 3.6 Assumptions and limitations . . . 11
- 3.7 Relation to FNO, U-FNO and attention . . . 11

4 Training and Results **12**
- 4.1 Datasets . . . 12
- 4.2 Training protocol . . . 12
- 4.3 Evaluation metrics . . . 12
- 4.4 Baselines . . . 12
- 4.5 Main results . . . 12
- 4.6 Ablations . . . 13

4.7 Discussion 13

A Notation **14**

1 Neural Operators and FNOs

1.1 Operator learning: problem setup

Neural Operators work by characterizing the inputs and outputs of its mapping as elements of function spaces. Let $f \in \mathcal{U}, g \in \mathcal{V}$ be functions defined on domains $D \subset \mathbb{R}^d$ and $D' \subset \mathbb{R}^{d'}$, respectively. The goal is to learn a mapping $G : U \to V$ such that $g = G(f)$. In practice, we observe these functions on discrete grids D_n and D'_m, leading to finite-dimensional representations of the functions.
The function space approach enables us to think of the mapping G as a continuous operator, suitable for the application of elements of functional analysis. This perspective allows for the design of architectures that are invariant to the discretization of the input and output functions, making them suitable for tasks where the underlying functions may be sampled at different resolutions. For example, in the context of solving PDEs, the input function f could represent initial conditions or parameters of the PDE, while the output function g could represent the solution of the PDE at a later time. The neural operator aims to learn this mapping in a way that is robust to changes in the discretization of the input and output functions.

1.2 Neural operators as stacked kernel integrals

One way of constructing a neural operator is with the approach of the Green's function of a linear operator. The Green's function $G(x, y)$ is the kernel of the integral operator that maps an input function f to an output function g via the integral

$$g(r) = \int_D \kappa(r, r', \theta) f(r')\, dr'. \tag{1}$$

With some external parameters θ. The choice of kernel $\kappa(r, r', \theta)$ determines the properties of the operator. A common approach is to assume that κ is translation invariant, meaning that

$$\kappa(r, r', \theta) = \kappa(r - r', \theta). \tag{2}$$

This assumption turns the operator effectively into a convolution, which can be efficiently computed in Fourier space. The kernel κ can be parameterized by a neural network, allowing the operator to learn complex mappings from input to output functions.

1.3 The Fourier Neural Operator

Since the kernel integral operator is translation invariant, it can be efficiently computed in Fourier space:

$$g(r) = \int_D \kappa(r - r', \theta) f(r')\, dr' = \mathcal{F}^{-1}\left[\mathcal{F}[\kappa] \cdot \mathcal{F}[f]\right](r), \tag{3}$$

The Fourier Neural Operator (FNO) leverages this approach in discrete function spaces, where the integral turns into a sum over discrete grid points:

$$g(r_i) = \sum_{j=1}^{N} \kappa(r_i - r_j, \theta) f(r_j) \Delta r_j, \tag{4}$$

where r_i and r_j are points on the discrete grid, and Δr_j is the grid spacing. The FNO computes this sum efficiently by transforming the input function f into Fourier space, applying the learned kernel κ in Fourier space, and then transforming back to physical space. This is achieved using the Fast Fourier Transform (FFT), which has a computational complexity of $\mathcal{O}(N \log N)$, making it suitable for large-scale problems. A diagram of the FNO architecture is shown in Figure 1. The FNO consists of three main components: a lifting layer that maps the input function f to a higher-dimensional representation, a series of Fourier layers that apply the learned kernel in Fourier space, and a projection layer that maps the output back to the desired function space. The lifting and projection layers are typically implemented as fully connected neural networks, while the Fourier layers consist of FFTs, learned multiplications in Fourier space, and inverse FFTs.

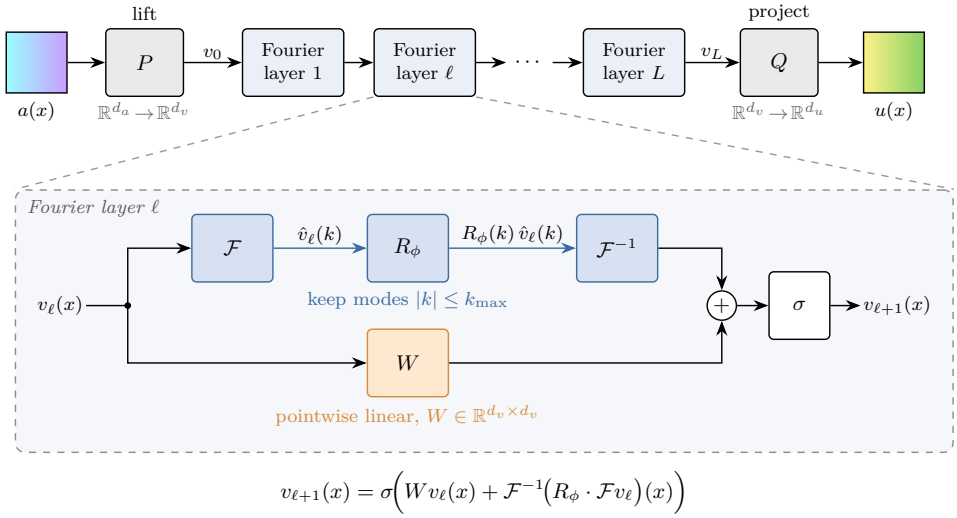


Figure 1: Fourier Neural Operator architecture.

While it is useful to think about FNOs from this theoretical framework of kernel integral operators in PDEs, there is no obstacle in removing that context and examining the architecture as a general-purpose operator learning framework. What happens on paper is that the input data is transformed into Fourier space, where a learned kernel is applied. The learned weights correspond to the Fourier coefficients of the kernel, so instead of spatial context, the FNO is learning a global, stationary, spectral representation of the data. This is a central tradeoff of the FNO architecture: it is very efficient and can learn global structure, but it is limited in its ability to learn local, multi-scale features.

1.4 Discretisation invariance and its limits

In practice, the FNO does not learn the kernel κ directly. Each Fourier layer stores the Fourier coefficients of the kernel for a fixed set of low wavenumbers and sets all others to zero:

$$\hat{g}(k) = \begin{cases} R_\phi(k)\,\hat{f}(k), & |k| \le k_{\max}, \\ 0, & |k| > k_{\max}, \end{cases} \tag{5}$$

where $\hat{f} = \mathcal{F}[f]$, and the $R_\phi(k) \in \mathbb{C}^{d_v \times d_v}$ are learned complex matrices that mix the d_v channels of the lifted representation. The parameters therefore live on wavenumbers, not on grid points:

their number is set by $k_{\max}$ and d_v and is independent of the number of grid points N. The same weights can be applied to an input sampled on any grid fine enough to resolve the retained modes, i.e. as long as the Nyquist wavenumber of the grid exceeds $k_{\max}$. The remaining components, namely the lifting P, the projection Q and the pointwise linear term W, act on each grid point separately and are likewise independent of the resolution. This is the sense in which the FNO is discretization invariant, and it enables the *zero-shot super-resolution* reported by Li et al. [3]: a model trained on a coarse grid can be evaluated on a finer grid without retraining.

This invariance is a property of the architecture, not of the learned map, and in practice it holds only approximately. Three caveats are relevant for this work.

Band-limited output. Since the spectral path only acts on $|k| \leq k_{\max}$, evaluating a trained FNO on a finer grid does not add information on the newly resolved scales. Any structure above $k_{\max}$ in the output has to be generated by the pointwise nonlinearity σ acting on a band-limited signal. Super-resolution with an FNO is therefore closer to a learned, nonlinear interpolation than to a prediction of genuinely new small-scale structure.

Aliasing. The nonlinearity σ creates harmonics above the band of its input. On a continuous domain these would simply be additional high-k modes; on a grid, everything above the Nyquist wavenumber is folded back onto lower wavenumbers. Which modes are folded where depends on N, so the same network evaluated on two different grids gives slightly different outputs, even on the shared low-k band. The discrete network is thus not an exact discretization of a single continuous operator. This aliasing error has been analysed systematically by Bartolucci et al. [1].

Resolution-dependent targets. Even a perfectly discretization invariant architecture cannot compensate for the fact that the data themselves change with resolution. If the training fields come from a simulation at a given cell size, the smoothing scales, sub-grid prescriptions and sharpness of features are tied to that cell size. For the ionization fields considered here, bubble boundaries are resolved down to the grid scale, so a field simulated at higher resolution contains genuinely different small-scale structure that a model trained at lower resolution has never seen.

Finally, the FFT assumes periodic boundary conditions. This is natural for coeval simulation boxes, which are periodic by construction, but not for lightcones, whose line-of-sight direction is not periodic. There, padding or windowing is needed; without it, the convolution wraps around and couples the two ends of the lightcone.

1.5 Limitations of a purely spectral kernel

The end of Section 1.3 identified the central tradeoff of the FNO: its kernel is global, stationary and spectral. Each of these properties is a strength for smooth, statistically homogeneous fields, and each becomes a limitation for fields with sharp, localized structure such as ionization maps.

Stationarity. Because κ depends only on $r - r'$, the spectral path applies the same linear response at every position. How strongly a point r' influences a point r cannot depend on the state of the field at either location. All state dependence has to enter through the pointwise terms and the nonlinearity σ, and builds up only by stacking layers. Many of the processes we want to emulate are not of this form: in reionization, whether ionizing photons from a source reach a given region depends on whether the gas in between is already ionized.

Mode truncation and sharp features. The truncation in Equation (5) limits each spectral path to wavelengths longer than roughly $L/k_{\max}$, where L is the box size. Sharp features have broad spectra: the Fourier coefficients of a step decay only as $1/k$, so much of the information that locates an edge sits at high wavenumbers. Truncating these modes produces the familiar smoothing and Gibbs-type ringing around discontinuities. The ionization field in reionization simulations is close to binary, with cells either neutral or ionized, so its bubble boundaries are exactly this kind of feature. The FNO has to reconstruct them from the pointwise path W and the nonlinearity, which only see a single grid cell.

No intermediate scales. The spectral path is global and the pointwise path is local to one cell, but nothing in between is represented explicitly. Every Fourier layer operates on the full grid with the same set of modes; there is no coarse-to-fine hierarchy and no local receptive field spanning a few cells. Structures at intermediate scales, such as individual bubbles a few to a few tens of cells across, have to be represented as combinations of global modes.

Cost of more modes. The obvious remedy, increasing $k_{\max}$, is expensive. Each retained mode carries a full $d_v \times d_v$ complex matrix, so the number of spectral parameters grows as $\mathcal{O}(k_{\max}^d\, d_v^2)$, i.e. cubically in $k_{\max}$ for three-dimensional fields. Keeping all modes also removes the implicit regularization that truncation provides.

These limitations concern not the accuracy of the FNO on large scales, where it typically performs well, but the fidelity of small-scale, phase-sensitive structure. Such errors can be hidden in the power spectrum, which discards phase information, and become visible in the cross-correlation coefficient $r(k)$ between prediction and truth (Equation (10)). They motivate architectures that build spatial structure into the operator explicitly, which are the subject of Section 2.

2 Spatial structure in neural operators and the U-Net

Section 1.5 showed that the FNO represents a field through global, stationary, band-limited modes plus a pointwise map. By *spatial structure* we mean everything this leaves out: locality (interactions that are strong between nearby points and weak between distant ones), an explicit hierarchy of scales, and interactions whose strength depends on where in the field they happen. Reionization fields need all three. Ionized bubbles span sizes from a few cells to a sizeable fraction of the box, their boundaries are sharp on the grid scale, and whether two regions interact depends on the ionization state between them. This section reviews how existing architectures build in such structure, with the U-Net as the main example, and identifies what none of them provides.

2.1 Inductive biases: locality, scale and stationarity

Each layer of a neural operator can be written in the kernel form of Equation (1). Architectures differ mainly in which kernels $\kappa(r, r', \theta)$ they allow, and these restrictions are their inductive biases. Three properties are useful for comparing them.

Receptive field. A convolutional neural network (CNN) uses kernels with compact support of a few grid cells, $\kappa(r - r') = 0$ for $|r - r'|$ larger than the stencil. A single layer is therefore strictly local; the receptive field grows only linearly with depth. The FNO is the opposite extreme: every Fourier layer couples every point to every other point.

Scale. A plain CNN or FNO processes the field at one resolution. Neither has a built-in notion of coarse and fine scales; scale-dependent behaviour has to be learned implicitly. Multi-scale architectures instead process the field at several resolutions and combine the results, so that large-scale context and small-scale detail are handled by separate parts of the network.

Stationarity and state dependence. CNNs, FNOs and U-Nets all use kernels that depend only on $r - r'$: the same filter is applied everywhere, and the coupling between two points is fixed once training ends. Attention mechanisms [6] and graph kernel networks [2] relax this by letting the kernel depend on the features at r and r', so that the coupling becomes *state-dependent*. The price is cost: a dense state-dependent kernel requires $\mathcal{O}(N^2)$ operations, which is prohibitive for three-dimensional grids with $N \sim 10^7$ cells.

Table 1 summarizes these properties for the architectures discussed in this report.

Table 1: Inductive biases of the operator architectures discussed in this report.

Architecture	Receptive field	Stationary	Multi-scale	State-dependent
CNN	local, grows with depth	yes	no	no
FNO	global (band-limited)	yes	no	no
U-Net	local $\to$ wide via pooling	yes	yes	no
U-FNO	global + multi-scale local	yes	yes	no
Attention	global	no	no	yes, $\mathcal{O}(N^2)$
Rhizome	global	no (gated)	implicit	yes, $\mathcal{O}(N \log N)$

2.2 The U-Net

The U-Net [5] was introduced for biomedical image segmentation, a task that, like predicting an ionization field, requires sharp, pixel-accurate boundaries informed by large-scale context. It consists of three parts (Figure 2).

Contracting path. The encoder applies a stack of small convolutions followed by a nonlinearity, then halves the resolution by pooling, and repeats. Denoting the feature map at level s by e_s,

$$e_0 = C_0(a), \qquad e_s = C_s\big(\mathrm{Down}(e_{s-1})\big), \quad s = 1, \dots, S, \tag{6}$$

where each C_s is a block of 3×3 convolutions with nonlinearities. The number of channels is typically doubled at each level, so the network trades spatial resolution for feature richness. Because every level operates on a grid that is coarser by a factor of two, a 3×3 convolution at level s spans $3 \cdot 2^s$ cells of the original grid. The receptive field therefore grows exponentially with the number of levels instead of linearly with depth.

Bottleneck. At the coarsest level, e_S, the field is represented on a grid of $N/2^S$ points per dimension with many channels. Here a few convolutions see a large fraction of the domain and capture large-scale context.

Expanding path and skip connections. The decoder upsamples step by step and, at each level, concatenates the upsampled features with the encoder features of the same resolution:

$$d_S = e_S, \qquad d_s = C'_s([\,\mathrm{Up}(d_{s+1}),\ e_s\,]), \quad s = S-1, \dots, 0, \tag{7}$$

and a final 1×1 convolution maps d_0 to the output u. The skip connections $e_s \to d_s$ are the key ingredient. Pooling discards the exact position of features, and upsampling alone cannot restore it. The skip connections pass the full-resolution encoder features directly to the decoder, so the network can combine *what* is present (from the coarse levels) with *where* it is (from the fine levels).

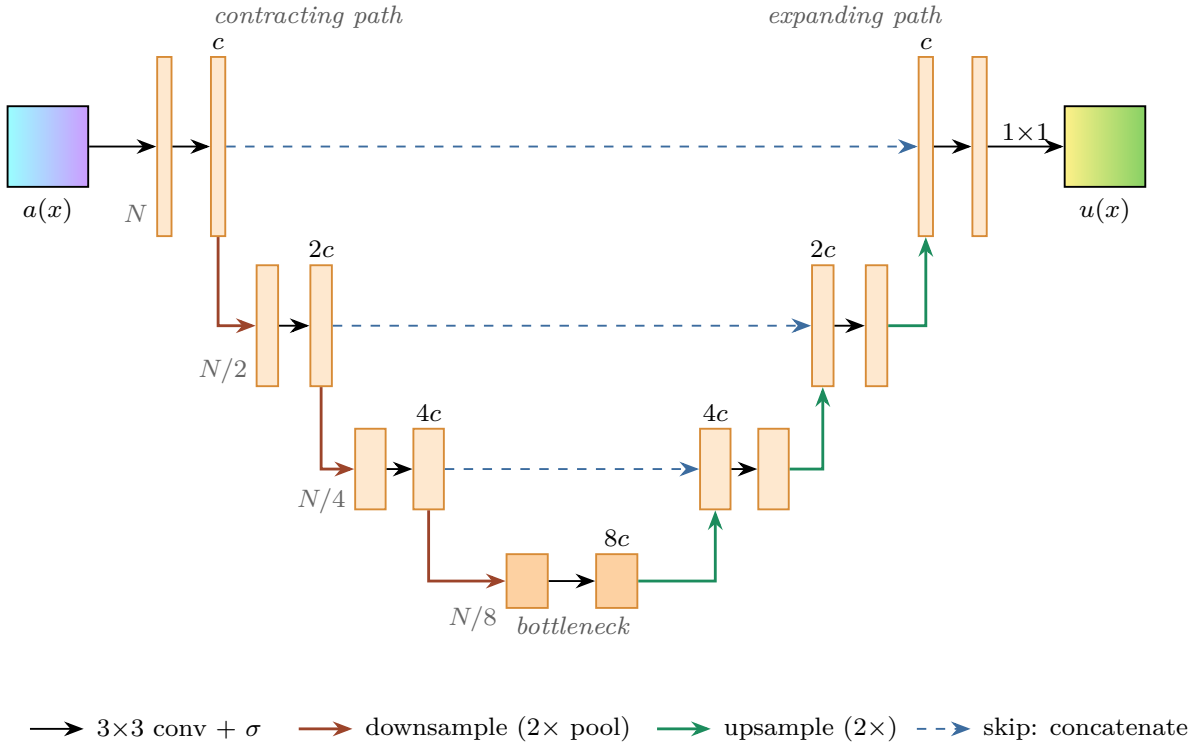


Figure 2: U-Net architecture with $S = 3$ downsampling steps. The contracting path (left) halves the resolution N and doubles the channel count c at each level; the expanding path (right) reverses this. Dashed skip connections concatenate encoder features with the upsampled decoder features of the same resolution, which restores the spatial detail lost by pooling.

Why it captures sharp structure. The U-Net addresses each limitation identified in Section 1.5. Its finest levels operate on the full grid with local kernels, so edges can be represented directly in physical space instead of through a truncated set of global modes. Its intermediate levels give it receptive fields at every scale between a few cells and the full box, which matches the broad range of bubble sizes. Finally, skip connections let high-frequency detail bypass the bottleneck entirely. U-Nets are therefore strong baselines for field-to-field emulation whenever sharp, localized features matter.

Limitations. The price is the loss of the properties that made the FNO attractive. The convolution kernels are defined in units of grid cells, not physical length, so changing the resolution changes the physical scale each filter responds to: the U-Net is not discretization

invariant, and a model trained at one resolution cannot be reused at another. The number of pooling steps also fixes which grid sizes are admissible (multiples of 2^S). Global context is only available at the bottleneck and only at coarse resolution, so long-range interactions are approximated rather than represented exactly. And the U-Net is still stationary: the same filters are applied everywhere, independent of the local state of the field.

2.3 Hybrid architectures: U-FNO and related

The complementary strengths of the FNO (global, resolution-independent) and the U-Net (local, multi-scale) suggest combining them. The U-FNO of Wen et al. [7] does this inside the Fourier layer. In addition to the spectral path $\mathcal{F}^{-1}(R_\phi \cdot \mathcal{F}v)$ and the pointwise path Wv of Figure 1, each *U-Fourier layer* adds a small U-Net acting on v:

$$v_{\ell+1}(x) = \sigma\Big(Wv_\ell(x) + \mathcal{F}^{-1}(R_\phi \cdot \mathcal{F}v_\ell)(x) + \mathrm{UNet}_\ell[v_\ell](x)\Big), \tag{8}$$

where UNet_ℓ denotes the U-Net path. The spectral path supplies global, low-k structure, while the U-Net path restores the high-k, localized detail that mode truncation removes. In the original application to multiphase flow, where saturation fronts are sharp, this substantially improved accuracy over the plain FNO [7].

The improvement comes at a cost that matters when comparing architectures. The U-Net path adds a large number of parameters, and a U-FNO is typically much larger than an FNO of the same width and depth. In our own comparison (Section 4), the U-FNO reference model has roughly two orders of magnitude more parameters than the matched models. Better accuracy can therefore not be attributed to the architecture alone. We treat the U-FNO as a dense reference baseline and make architectural claims only between parameter-matched models. The U-FNO also inherits the U-Net's resolution dependence through its convolutional path, and remains stationary.

Other hybrids follow the same pattern of adding a local or multi-scale branch next to a global spectral one. They differ in which branch is used and where it is inserted, but they share the structure of Equation (8): a sum of fixed, stationary operators.

2.4 Open problem: structure that adapts to the field

All architectures above, apart from attention, share one property: the communication pattern between points is fixed once training is done. Whether a kernel is local or global, single- or multi-scale, the coupling between r and r' depends only on their separation (and the level in the hierarchy), never on the field itself. Any dependence on the current state has to be built up indirectly, through pointwise nonlinearities stacked over many layers.

For reionization, this is at odds with the physics. Ionizing photons from a source travel until they are absorbed, so whether a source at r' affects a point r depends on the neutral fraction along the way: through an ionized region the coupling is long-range, through neutral gas it is short. The relevant interaction kernel is therefore state-dependent. Attention can represent this but scales as $\mathcal{O}(N^2)$; the stationary architectures scale well but cannot represent it directly.

An operator suited to this problem should combine four properties: (i) global reach, so that interactions across the whole box are possible; (ii) state-dependent coupling, so that the effective connections change as the field evolves; (iii) a cost close to $\mathcal{O}(N \log N)$, so that three-dimensional

grids remain tractable; and (iv) a parameterization that is independent of the grid, so that the discretization invariance of the FNO is kept. The rhizome operator, introduced in Section 3, is designed around these four requirements.

3 Rhizome Neural Operator approach

[**TODO:** Intro and the rhizome metaphor: every cell can connect to every other, and the strength of those connections is set by the current state rather than fixed by a hierarchy.]

3.1 Design principles

[**TODO:** Single interaction mechanism (no separate local/global branches); connections are a state-dependent integral operator; weight-shared recurrence.]

3.2 State-dependent integral interaction

At iteration t, cell i at position r_i receives the message

$$\boldsymbol{m}_i^t = \frac{1}{N} \sum_{j=1}^{N} \operatorname{diag}\big(A_\theta(\boldsymbol{h}_i^t, \boldsymbol{c}_i)\big)\, \kappa_\theta(r_i - r_j)\, \operatorname{diag}\big(B_\theta(\boldsymbol{h}_j^t, \boldsymbol{c}_j)\big)\, V_\theta(\boldsymbol{h}_j^t, \boldsymbol{c}_j), \tag{9}$$

where

- B_θ is the learned *source* (transmission) gate,
- A_θ is the learned *receiver* (response) gate,
- κ_θ is a learned periodic, channel-mixing spatial kernel,
- V_θ builds the transmitted value from state and conditioning,
- $N = HW$ is uniform quadrature on the normalised periodic box.

[**TODO:** Continuum form of (9) as an integral over $\mathbb{T}^d$; show it reduces to the FNO kernel when $A_\theta = B_\theta = 1$.]

Proposition 3.1 (Reduction to a Fourier layer)**.** ***[TODO:** State and prove.]*

3.3 Gating: source and receiver factors

[**TODO:** Gate range $(0, 2)$ via $2\,\mathrm{sigmoid}$; recomputed every iteration; channel-wise factorisation avoids an $N \times N$ edge tensor; signed and asymmetric interactions allowed.]

3.4 Recurrent time-stepping

Algorithm 1 Rhizome operator forward pass

Require: density bands δ, redshift z, parameters ϑ

$\boldsymbol{c} \leftarrow \mathrm{lift}(\delta) + \mathrm{embed}(z, \vartheta)$

$\boldsymbol{h}^0 \leftarrow \tanh(\boldsymbol{c})$

for $t = 0, \ldots, T-1$ **do** ▷ shared weights

  $(\boldsymbol{s}, \boldsymbol{r}) \leftarrow 2\,\mathrm{sigmoid}(\mathrm{gates}([\boldsymbol{h}^t, \boldsymbol{c}]))$

  $\boldsymbol{v} \leftarrow V_\theta([\boldsymbol{h}^t, \boldsymbol{c}])$

  $\boldsymbol{m}^t \leftarrow \boldsymbol{r} \odot \mathcal{F}^{-1}(R \cdot \mathcal{F}(\boldsymbol{s} \odot \boldsymbol{v}))$

  $\boldsymbol{p} \leftarrow \tanh\left(\mathrm{pointwise}([\boldsymbol{h}^t, \boldsymbol{m}^t, \boldsymbol{c}])\right)$

  $\boldsymbol{h}^{t+1} \leftarrow (1 - \Delta t)\,\boldsymbol{h}^t + \Delta t\,\boldsymbol{p}$

end for

return $x_{\mathrm{HI}} = \mathrm{sigmoid}(\mathrm{decode}(\boldsymbol{h}^T))$

[**TODO:** Interpretation as explicit Euler integration; stability in Δt; choice of T.]

3.5 Discretisation and spectral evaluation

[**TODO:** FFT is an efficient evaluator of the connections, not a separate predictor; orthonormal FFT pair already gives the $1/N$ quadrature; real DC and conjugate-frequency constraints for a real kernel.]

3.6 Assumptions and limitations

Assumption 3.2. [**TODO:** e.g. periodic domain; translation-invariant base kernel.]

[**TODO:** No reciprocity, conservation, or radiative-transfer physics is imposed; expressivity vs. a fully pair-dependent kernel.]

3.7 Relation to FNO, U-FNO and attention

[**TODO:** Position against Table 1; parameter count and cost $\mathcal{O}(T\,N \log N)$.]

Figure placeholder:
`rhizome_architecture.pdf`
Rhizome block diagram: gates, value map, spectral kernel, recurrence.

Figure 3: Rhizome Neural Operator architecture.

4 Training and Results

4.1 Datasets

[**TODO:** Darcy-flow benchmark; synthetic lightcones (toy pilot); `21cmFAST` ionisation fields [4]. Resolutions, splits, conditioning variables.]

Table 2: Datasets used in this report.

Dataset	Grid	Train / Val / Test	Target
Darcy flow	[**TODO:**]	[**TODO:**]	[**TODO:**]
Synthetic lightcones	[**TODO:**]	[**TODO:**]	x_{HI}
`21cmFAST`	[**TODO:**]	[**TODO:**]	x_{HI}

4.2 Training protocol

[**TODO:** Loss, optimiser, LR schedule with validation-driven reductions, predefined plateau stopping rule, checkpointing and seeds.]

4.3 Evaluation metrics

[**TODO:** Relative L^2 error; power spectrum ratio; cross-correlation $r(k)$; bubble-size distribution; global neutral fraction.]

$$r(k) = \frac{P_{\mathrm{pred}\times\mathrm{true}}(k)}{\sqrt{P_{\mathrm{pred}}(k)\,P_{\mathrm{true}}(k)}} \tag{10}$$

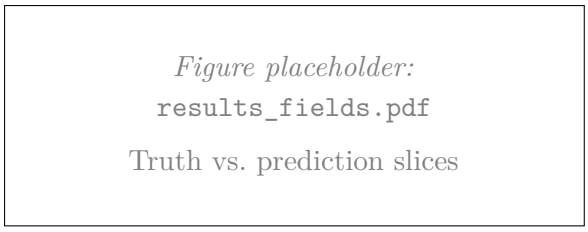
4.4 Baselines

[**TODO:** FNO, U-FNO, U-Net, recurrent FNO; parameter-matched comparisons.]

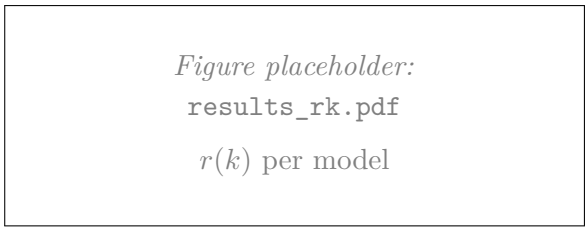
4.5 Main results

Table 3: Held-out performance. [**TODO:** fill in]

Model	Params	Rel. $L^2 \downarrow$	$r(k_*) \uparrow$	BSD bias $\downarrow$
FNO	[**TODO:**]	[**TODO:**]	[**TODO:**]	[**TODO:**]
U-FNO	[**TODO:**]	[**TODO:**]	[**TODO:**]	[**TODO:**]
U-Net	[**TODO:**]	[**TODO:**]	[**TODO:**]	[**TODO:**]
Rhizome	[**TODO:**]	[**TODO:**]	[**TODO:**]	[**TODO:**]



(a) Field comparison



(b) Cross-correlation

Figure 4: Qualitative and spectral comparison. **[TODO:]**

4.6 Ablations

[TODO: Stage A: gate mechanism. Stage B: weight sharing and FNO reduction. Stage C: operator transfer and 21 cm validation. Decision rules stated before results.]

4.7 Discussion

[TODO: What the gates learn; where the rhizome helps and where it does not; resolution transfer; next steps.]

A Notation

Symbol	Meaning
$\mathcal{G},\ \mathcal{G}_\theta$	true / learned operator
$\mathcal{A},\ \mathcal{U}$	input / output function spaces
$\mathcal{K}$	kernel integral operator
$\mathcal{F},\ \mathcal{F}^{-1}$	Fourier transform and inverse
$k_{\max}$	number of retained Fourier modes
d_v	channel width
$\boldsymbol{h}^t,\ \boldsymbol{c},\ \boldsymbol{m}^t$	hidden state, conditioning, message
$A_\theta,\ B_\theta$	receiver / source gates
$V_\theta,\ \kappa_\theta$	value map, spatial kernel
$T,\ \Delta t$	number of iterations, step size
x_{HI}	neutral hydrogen fraction

References

[1] Francesca Bartolucci, Emmanuel de Bézenac, Bogdan Raonić, Roberto Molinaro, Siddhartha Mishra, and Rima Alaifari. Representation equivalent neural operators: a framework for alias-free operator learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

[2] Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Neural operator: Graph kernel network for partial differential equations. *arXiv preprint arXiv:2003.03485*, 2020.

[3] Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations (ICLR)*, 2021.

[4] Andrei Mesinger, Steven Furlanetto, and Renyue Cen. 21cmfast: a fast, seminumerical simulation of the high-redshift 21-cm signal. *Monthly Notices of the Royal Astronomical Society*, 411(2):955–972, 2011.

[5] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, pages 234–241, 2015.

[6] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.

[7] Gege Wen, Zongyi Li, Kamyar Azizzadenesheli, Anima Anandkumar, and Sally M. Benson. U-fno—an enhanced fourier neural operator-based deep-learning model for multiphase flow. *Advances in Water Resources*, 163:104180, 2022.